

# PROJECT REPORT

## STOCK TRADING APP

Project name: SB STOCKS

Project Submitted to

APSCHE

Submitted By:

Gatiganti Sai Surya Lakshmi  
Radhika Devi  
Tanuku N V S Surya Prasad  
Gedela Adi Chakradhar  
Anudeep Gudala

# STOCK TRADING APPLICATION

## INTRODUCTION :

SB Stocks is an innovative stock trading web application designed to provide users with a realistic and risk-free environment to learn and practice stock market trading. The platform enables users to explore real-time US stock market data, analyze historical stock trends, and perform virtual buying and selling of stocks using paper trading features. By offering a simple and interactive interface, SB Stocks helps beginners understand trading concepts while allowing experienced users to test different investment strategies without financial loss.

The application provides essential functionalities such as user registration, secure authentication, stock browsing, portfolio management, watchlist creation, and transaction tracking. Users can monitor their investments through a personalized dashboard that displays portfolio value, holdings, cash balance, profit/loss analysis, and overall trading performance. With access to detailed stock information and market insights, users can make informed decisions and improve their trading knowledge.

Built using the MERN stack architecture, SB Stocks combines React.js for a responsive frontend, Node.js and Express.js for efficient backend operations, and MongoDB for secure and scalable data storage. The system follows a structured client-server model to ensure smooth communication between the user interface and backend services. By integrating modern web technologies with stock simulation features, SB Stocks delivers an engaging, educational, and practical platform for users to enhance their understanding of the financial market.

## KEY FEATURES :

### USER REGISTRATION & AUTHENTICATION :

- **SECURE SIGN-UP** using username, email, and password authentication to create a personalized trading account.
- **USER PROFILE MANAGEMENT** securely maintains user details, account information, virtual funds, portfolio data, and trading activities for future access.

### REAL-TIME STOCK MARKET DATA :

- **PROVIDES LIVE STOCK INFORMATION** by displaying real-time US stock market data including stock prices, company details, and market movements.

- **HISTORICAL DATA AND MARKET TRENDS** allow users to analyze previous stock performance, understand price changes, and make better trading decisions.

## **STOCK EXPLORATION & FILTERING :**

- **ALLOWS USERS TO SEARCH AND FILTER STOCKS** based on company names, symbols, and market information to quickly find required stocks.
- **DETAILED STOCK ANALYSIS** provides users with important financial information and price updates before performing trading actions.

## **VIRTUAL TRADING & PORTFOLIO MANAGEMENT :**

- **PAPER TRADING FUNCTIONALITY** allows users to buy and sell stocks using virtual money without involving any real financial risk.
- **PORTFOLIO TRACKING SYSTEM** helps users monitor holdings, total investment value, available balance, profits, losses, and overall trading performance.

## **PERSONALIZED DASHBOARD :**

- **INTERACTIVE DASHBOARD INTERFACE** displays important trading information such as portfolio value, cash balance, stock holdings, and investment summaries.
- **VISUAL PERFORMANCE TRACKING** helps users understand their trading progress and evaluate investment strategies effectively.

## **WATCHLIST MANAGEMENT :**

- **SAVE FAVOURITE STOCKS** feature allows users to add important stocks into a watchlist for quick access and continuous monitoring.
- **EASY MARKET TRACKING** helps traders follow selected stocks and observe price changes without searching repeatedly.

## **TRANSACTION MANAGEMENT :**

- **COMPLETE TRADING HISTORY** stores every buy and sell operation performed by users with details such as stock name, quantity, price, and date.
- **TRANSACTION RECORD ANALYSIS** helps users review previous trading decisions and improve future investment strategies.

## **USER-FRIENDLY INTERFACE :**

- **RESPONSIVE AND MODERN DESIGN** provides a clean and simple interface accessible across different devices.
- **SMOOTH USER EXPERIENCE** ensures easy navigation between dashboard, stocks, portfolio, watchlist, and transaction sections for effective trading simulation.

## **DESCRIPTION :**

The SB Stocks App is a user-centric stock trading platform designed to make learning and practicing stock market trading simple, interactive, and risk-free. The application provides a digital paper trading environment where users can explore real-time US stock market data, analyze stock trends, and perform virtual buying and selling of stocks through an easy-to-use interface. It helps users understand market movements, improve trading knowledge, and test investment strategies without using real money.

For users, the app offers secure registration, login authentication, personalized dashboards, and virtual portfolio management features. Users can browse different stocks, view market information, add stocks to their watchlist, perform simulated trades, and monitor portfolio performance including holdings, investments, profits, and losses. The transaction management system records all buying and selling activities, allowing users to review their trading history and improve decision-making skills.

With features that enhance financial learning, market analysis, and strategy development, the SB Stocks App provides a complete virtual trading experience. It supports beginners who are learning the fundamentals of stock markets and experienced users who want to experiment with different trading strategies, creating a reliable and educational platform for improving stock trading skills.

## **SCENARIO-BASED CASE STUDY :**

### **1. USER REGISTRATION :**

Alex, a beginner interested in learning stock market trading, opens the SB Stocks application. He starts by creating a new account by providing basic details such as username, email address, and password. The application securely stores his information and creates a personalized trading profile. After successful registration, Alex can log in and access the stock trading dashboard.

### **2. USER LOGIN AND DASHBOARD ACCESS :**

After logging into SB Stocks, Alex is redirected to his personalized dashboard. The dashboard displays important information such as total portfolio value, available virtual cash balance, investment summary, asset allocation, and overall trading performance. This helps Alex easily understand his current trading activities and portfolio status.

### **3. BROWSING AND ANALYZING STOCKS :**

Alex visits the stock market section where he can explore different US stocks available on the platform. The application allows him to search and filter stocks based on company names and stock symbols. He analyzes real-time stock prices, market trends, and company information before making trading decisions.

#### **4. ADDING STOCKS TO WATCHLIST :**

While browsing the market, Alex finds some companies that he wants to monitor regularly. He adds these stocks to his watchlist for quick access. The watchlist feature allows him to track selected stocks, observe price changes, and analyze future investment opportunities without searching repeatedly.

#### **5. VIRTUAL STOCK TRADING :**

After analyzing a stock, Alex decides to perform a simulated trade. He selects the stock, enters the required quantity, and completes a virtual buying process using paper trading funds. The application updates his portfolio automatically by adding the purchased stocks and adjusting the available balance.

#### **6. PORTFOLIO MANAGEMENT :**

After completing trades, Alex accesses the portfolio section to monitor his investments. The portfolio dashboard displays details such as owned stocks, number of shares, total investment amount, current value, profit/loss percentage, and overall portfolio performance. This helps him evaluate his trading strategies effectively.

#### **7. SELLING STOCKS AND UPDATING HOLDINGS :**

When Alex wants to test an exit strategy, he sells selected stocks from his portfolio. The system processes the sell transaction, updates his holdings, adjusts the virtual cash balance, and recalculates the overall portfolio value based on the latest stock information.

#### **8. TRANSACTION HISTORY MANAGEMENT :**

Every buying and selling activity performed by Alex is recorded in the transaction history section. He can view complete details including stock name, transaction type, quantity, price, and date. This helps him review previous decisions and improve future trading strategies.

#### **9. MARKET ANALYSIS AND STRATEGY IMPROVEMENT :**

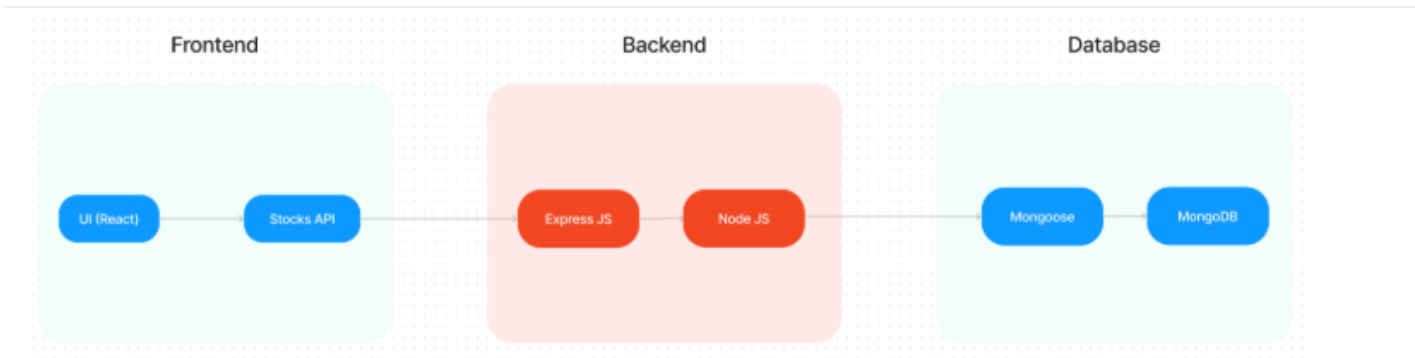
Using stock information, historical trends, portfolio performance, and transaction records, Alex analyzes his trading behavior. The application helps him understand market movements, compare strategies, and improve decision-making skills without any financial risk.

**10. CONTINUOUS LEARNING AND TRADING PRACTICE :**

By regularly using SB Stocks, Alex gains practical knowledge about stock trading and investment management. The platform provides a safe environment for beginners and experienced traders to practice strategies, improve confidence, and prepare for real-world stock market participation

**TECHNICAL ARCHITECTURE :**

The SB Stocks App features a modern client-server architecture designed to provide a smooth and secure stock trading simulation experience. The frontend is developed using React.js to create a responsive and interactive user interface, while API communication connects the client with the backend services. The backend is powered by Node.js and Express.js, which handle user authentication, stock operations, portfolio management, watchlist updates, and transaction processing. MongoDB provides scalable data storage for user details, stock information, holdings, and trading records. The application follows an MVC architecture to maintain organized code structure, improve performance, and support future scalability.



**FRONTEND TECHNOLOGIES :**

React.js: Used to build a responsive and interactive user interface for stock browsing, portfolio management, and dashboards.

Tailwind CSS: Provides modern styling and ensures a clean, user-friendly design across different devices.

Axios: Handles communication between the frontend and backend through API requests.

## **BACKEND FRAMEWORK :**

Express.js: A Node.js framework used to handle server-side logic, API routing, and request management for stock trading operations.

## **DATABASE AND AUTHENTICATION :**

MongoDB: A NoSQL database used to store user details, stock data, portfolios, watchlists, and transaction records.

JWT: Provides secure user authentication and manages user sessions.

Bcrypt: Encrypts user passwords to ensure secure data storage.

## **PORTFOLIO MANAGEMENT :**

Dashboard Interface: Allows users to view portfolio value, holdings, balance, and trading performance.

Transaction Management: Maintains records of all virtual buy and sell stock activities.

## **SCALABILITY AND PERFORMANCE :**

MongoDB: Supports large amounts of data and provides efficient storage as the application grows.

Optimized APIs: Improve communication speed between frontend, backend, and database.

## **SECURITY FEATURES :**

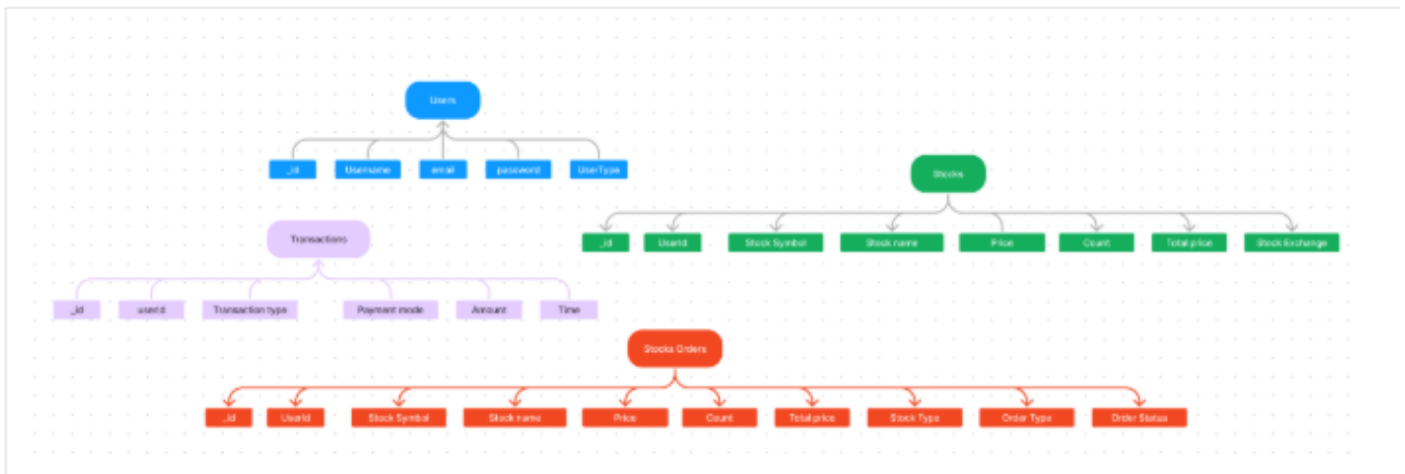
HTTPS: Secures data transfer between the client and server.

Data Protection: Protects user information and trading records using authentication and encryption methods.

## **WATCHLIST MANAGEMENT :**

Watchlist: Allows users to save and monitor their favourite stocks for quick access and future analysis.

## ER DIAGRAM :



The Entity-Relationship (ER) diagram for the SB Stocks application represents four key entities: Users, Stocks, Transactions, and Stock Orders, with their respective attributes and relationships.

The Users collection holds basic user information, including `_id`, `username`, `email`, `password`, and `userType`. Each user has a unique account in the system that allows them to access the trading platform, manage portfolios, perform transactions, and monitor stock activities securely.

The Stocks collection stores information related to available stocks, such as `_id`, `userId`, `stock symbol`, `stock name`, `price`, `count`, `total price`, and `stock exchange`. This collection maintains details about different stocks available in the market and provides users with information required for virtual trading and investment analysis.

The Transactions collection stores details about user trading activities, including `_id`, `userId`, `transaction type`, `payment mode`, `amount`, and `time`. This collection records all buy and sell operations performed by users, helping them track their complete trading history and analyze previous investment decisions.

The Stock Orders collection maintains information about stock orders, including `_id`, `userId`, `stock symbol`, `stock name`, `price`, `count`, `total price`, `stock type`, `order type`, and `order status`. It manages the relationship between users and stocks by recording every virtual trading operation performed within the application.

The relationships are as follows: one User can have multiple Transactions (one-to-many), one User can create multiple Stock Orders (one-to-many), and each Stock Order is associated with specific Stock details. The `userId` field connects users with their transactions and stock



activities, enabling SB Stocks to efficiently manage portfolio tracking, virtual trading operations, and user investment performance.

## **PRE REQUISITES :**

### **NODE.JS AND NPM :**

Node.js is a JavaScript runtime environment that allows JavaScript code to run on the server side. It helps in building fast and scalable backend applications.

npm (Node Package Manager) is used to install required libraries, manage dependencies, and maintain project packages.

Run npm init to create a package.json file and set up the backend project environment.

### **EXPRESS.JS :**

Express.js is a lightweight web framework for Node.js used to build APIs and manage server-side operations.

It handles routing, middleware, HTTP requests, and backend functionality for stock trading operations.

Install Express:

```
npm install express
```

### **MONGODB :**

MongoDB is a NoSQL database used to store application data in a flexible document format.

In SB Stocks, MongoDB stores user details, stock information, portfolios, watchlists, and transaction records.

It provides fast data retrieval and supports scalable application development.

### **REACT.JS :**

React.js is a JavaScript library used for creating interactive and reusable frontend components.

In SB Stocks, React.js manages the user interface including dashboard, stock market page, portfolio, watchlist, and transaction history pages.

React provides a smooth and dynamic user experience.

### **TAILWIND CSS :**

Tailwind CSS is a styling framework used to design modern and responsive user interfaces.

It helps create attractive layouts, dashboards, buttons, and components with better user experience.

### **HTML, CSS, AND JAVASCRIPT :**

Basic knowledge of HTML, CSS, and JavaScript is required to create the structure, styling, and functionality of the web application.

They are used along with React.js to build interactive frontend pages.

### **DATABASE CONNECTIVITY (MONGOOSE) :**

Mongoose is an Object Data Modeling (ODM) library used to connect Node.js applications with MongoDB.

It helps create database schemas and manage CRUD operations for users, stocks, portfolios, and transactions.

### **API COMMUNICATION (AXIOS) :**

Axios is used to send HTTP requests between the frontend and backend.

It helps transfer stock data, user details, and transaction information smoothly between React.js and Express.js.

### **SETUP AND INSTALLATION INSTRUCTIONS :**

#### **CLONE THE PROJECT REPOSITORY :**

Download the SB Stocks project files from GitHub or clone the repository using Git.

#### **INSTALL DEPENDENCIES :**

Navigate to both frontend and backend folders and install the required dependencies.

Frontend:

Move to the client folder and run:

```
npm install
```

Backend:

Move to the server folder and run:

```
npm install
```

#### **START THE DEVELOPMENT SERVER :**

After installing dependencies, start both frontend and backend servers.

Frontend:

```
npm run dev
```

Frontend will run on:

```
http://localhost:3000
```

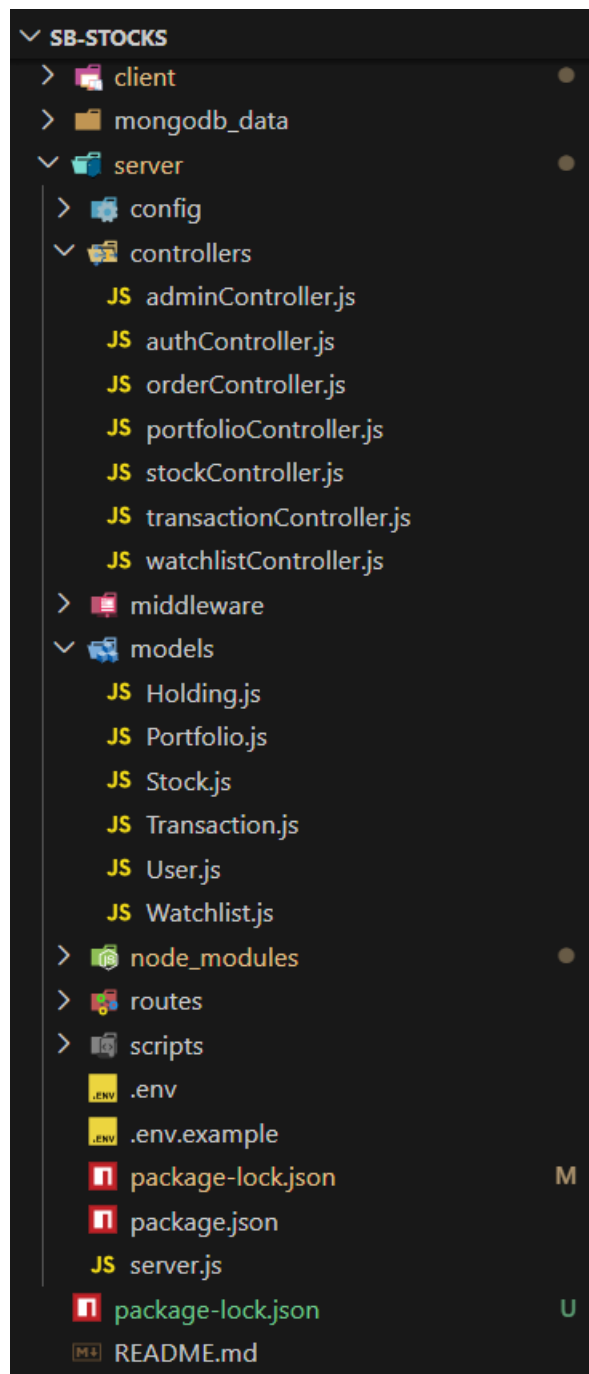
Backend:

```
nodemon index.js
```

Backend will run on:

```
http://localhost:5143
```

## PROJECT STRUCTURE :



The project is structured to include both Frontend and Backend components, each responsible for specific tasks in the development of the SB Stocks application. The frontend manages the user interface, allowing users to browse stocks, manage portfolios, monitor watchlists, and view transaction history. The backend handles server-side operations such as user authentication, stock data management, portfolio updates, and transaction processing. Below is the flow describing the roles and responsibilities of users and the system within the stock trading platform.

## **FRONTEND PART :**

The frontend is responsible for creating and displaying the user interface that traders interact with in the SB Stocks application. It provides a smooth experience for users to browse stocks, manage portfolios, track transactions, and monitor their investments. The frontend consists of the following components:

**REACT COMPONENTS** – Each component is designed for specific user interactions such as displaying stock lists, portfolio details, watchlists, dashboards, and transaction history.

**ROUTING** – Handles navigation between different pages like login, dashboard, stock market, portfolio, watchlist, and transaction history pages.

**STATE MANAGEMENT** – Maintains user session details, stock information, portfolio values, holdings, and trading updates throughout the application.

**STYLING** – Uses Tailwind CSS and modern design techniques to create responsive and user-friendly components.

## **BACKEND PART :**

The backend manages all server-side operations including user authentication, database communication, API handling, and trading-related operations. It contains the following modules:

**API ENDPOINTS** – Defines routes for handling user activities such as login, stock fetching, portfolio updates, watchlist management, and transaction processing.

**DATABASE MODELS** – Defines schemas for Users, Stocks, Portfolio, Watchlist, and Transactions using MongoDB and Mongoose.

**AUTHENTICATION & AUTHORIZATION** – Manages secure user registration, login verification, and protected access to user data.

**PORTFOLIO MANAGEMENT SYSTEM** – Handles stock buying, selling, holdings calculation, profit/loss tracking, and investment updates.

**TRANSACTION SYSTEM** – Stores and manages complete records of user trading activities.

## **APPLICATION FLOW :**

This project mainly focuses on stock traders/users who interact with different modules of the SB Stocks platform. Each functionality is managed through frontend interfaces and backend API services.

## **USER :**

**CREATE AN ACCOUNT & LOGIN** – Users can register and securely login using their email and password.

**VIEW STOCKS** – After logging in, users can explore available stocks with company information and market details.

**MANAGE WATCHLIST** – Users can add selected stocks to their watchlist for quick access and future monitoring.

**BUY AND SELL STOCKS** – Users can perform virtual stock trading using paper trading funds without real financial risk.

**VIEW PORTFOLIO** – Users can track stock holdings, investment value, available balance, profit/loss, and overall portfolio performance.

**VIEW TRANSACTION HISTORY** – Users can check previous buy and sell activities including stock details, quantity, price, and date.

**ANALYZE PERFORMANCE** – Users can evaluate trading strategies using portfolio data and transaction records to improve decision-making skills.

```
{  You, yesterday • sb-stocks ...
  "name": "sb-stocks-server",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "sb-stocks-server",
      "version": "1.0.0",
      "license": "ISC",
      "dependencies": {
        "bcryptjs": "^2.4.3",
        "cors": "^2.8.5",
        "dotenv": "^16.4.5",
        "express": "^4.19.2",
        "express-rate-limit": "^7.2.0",
        "express-validator": "^7.0.1",
        "helmet": "^7.1.0",
        "jsonwebtoken": "^9.0.2",
        "mongo-sanitize": "^1.1.0",
        "mongoose": "^8.3.1"
      },
      "devDependencies": {
        "nodemon": "^3.1.0"
      }
    }
  },
}
```

## **PROJECT FOLDERS :**

Frontend Folder: Contains all code related to the user interface, developed using JavaScript with React.js and modern frontend technologies. It manages dashboards, stock pages, portfolio views, watchlists, and transaction screens. This structure helps separate user interface logic from backend operations.

Backend Folder: Handles server-side functionality, API routes, authentication, database operations, and trading logic. It is developed using Node.js and Express.js. Maintaining separate frontend and backend folders improves project organization, scalability, and easy maintenance.

## **LIBRARY AND TOOL INSTALLATION :**

### **Backend Libraries :**

Node.js: Provides a runtime environment to execute JavaScript code on the server side and build scalable backend applications.

Express.js: A lightweight backend framework used for creating APIs, managing routes, and handling client-server communication.

MongoDB: A NoSQL database used for flexible storage of user details, stock data, portfolios, watchlists, and transaction records.

Mongoose: Provides database connectivity between Node.js and MongoDB, allowing easy schema creation and CRUD operations.

Bcrypt: Encrypts user passwords for secure authentication and protects sensitive user information.

JWT: Provides secure user authentication and manages access to protected routes.

### **Frontend Libraries :**

React.js: Manages component-based UI development and allows creation of reusable components for dashboard, stocks, portfolio, and watchlist pages.

Tailwind CSS: Provides responsive styling and helps create a clean, modern, and user-friendly interface.

Axios: Handles HTTP requests and allows smooth communication between the React frontend and Express backend.

## **MILESTONE 2: BACKEND DEVELOPMENT :**

The backend forms the core logic and data management system of the SB Stocks application. A well-structured backend ensures secure user authentication, efficient stock data handling, portfolio management, transaction processing, and smooth communication between the frontend and database. It provides scalability, security, and reliability for the overall stock trading platform.

## **EXPRESS.JS SERVER SETUP :**

Express Server: Acts as the central server for handling all requests and responses between the frontend and backend. It processes user actions such as login, stock browsing, portfolio updates, watchlist changes, and transaction requests.

Middleware Configuration: Middleware such as cors and body-parser allows smooth communication between frontend and backend applications. Middleware also helps in request processing, authentication checking, error handling, and data validation without affecting the main application logic.

## **API ROUTE DEFINITION :**

Route Organization: Organizing routes based on functionality such as authentication, stocks, portfolio, transactions, and watchlist keeps the code structure clean and easy to maintain. Each route file manages a specific part of the SB Stocks application.

Express Route Handlers: Route handlers manage data flow between the client and server by performing operations such as fetching stock details, updating portfolios, storing transactions, and managing user information.

## **DATA MODELS (SCHEMAS) WITH MONGOOSE :**

User Schema: Defines the structure for storing user information such as username, email, password, and account details. It ensures secure and organized user data management.

Stock and Portfolio Models: These schemas store stock information, user holdings, investment details, and portfolio performance data, allowing users to manage their virtual trading activities efficiently.

Transaction and Watchlist Models: These models maintain records of buy/sell activities and saved stocks, helping users track trading history and monitor favourite stocks.

CRUD Operations: CRUD functionalities (Create, Read, Update, Delete) allow efficient management of users, stocks, portfolios, watchlists, and transaction data.

## **USER AUTHENTICATION :**

JWT Authentication: JSON Web Tokens securely manage user sessions and ensure that only authorized users can access protected features such as portfolios, transactions, and watchlists.

## **STOCK AND TRANSACTION HANDLING :**

Stock Management: Handles stock information, price details, and user interactions with available stocks on the platform.

Transaction Management: Allows users to perform virtual buy and sell operations, update portfolios, and maintain complete trading history records.

## **ERROR HANDLING :**

Middleware for Error Handling: Error-handling middleware detects application issues and provides proper error responses for failed requests, improving debugging and user experience.

## **MILESTONE 3: DATABASE DEVELOPMENT :**

Using MongoDB as the database provides a flexible and scalable structure for managing different types of stock trading data including users, stocks, portfolios, watchlists, and transactions.

### **SCHEMAS FOR DATABASE COLLECTIONS :**

User Schema: Defines user information fields like username, email, password, and authentication details for secure account management.

Stock Schema: Stores stock-related information such as stock name, symbol, price, quantity, and market details.

Portfolio Schema: Maintains user investment details, stock holdings, portfolio value, and performance information.

Transaction Schema: Stores complete buy and sell records including stock details, quantity, price, and transaction type.

Watchlist Schema: Stores user-selected stocks for quick access and continuous monitoring.

### **DATABASE COLLECTIONS IN MONGODB :**

MongoDB collections such as users, stocks, portfolios, transactions, and watchlists provide a structured NoSQL approach for managing data efficiently and supporting application scalability.

## **MILESTONE 4: FRONTEND DEVELOPMENT :**

Frontend development focuses on building an interactive and user-friendly stock trading interface using React.js components. It provides users with smooth navigation and real-time interaction with trading features.



## **REACT APPLICATION SETUP :**

Folder Structure and Libraries: Creating a structured React project with organized folders for components, pages, and services improves development workflow and makes the application easy to maintain.

UI Libraries and Styling: Tailwind CSS and responsive design techniques help create a clean, modern, and visually appealing interface across all screens.

## **UI COMPONENTS FOR REUSABILITY :**

Reusable Components: Different UI elements such as stock cards, dashboards, buttons, forms, portfolio sections, and watchlist components are developed as reusable components to reduce development time.

Styling and Layout: Proper styling and layout management provide a consistent design, easy navigation, and a better user experience throughout the SB Stocks application.

## **LANDING PAGE :**



SIGN IN TO YOUR PAPER PORTFOLIO

## Welcome Back

EMAIL ADDRESS

 name@example.com

PASSWORD

 .....

**Sign In** →

Don't have an account? [Create Account](#)

### Demo Accounts:

User: [john@example.com](#) | Pass: [password123](#)

Admin: [admin@example.com](#) | Pass: [admin123](#)

REGISTRATION  
PAGE :



JOIN THE PAPER STOCK MARKET

## Create Account

FULL NAME

 John Doe

EMAIL ADDRESS

 john@example.com

PASSWORD

 At least 6 characters

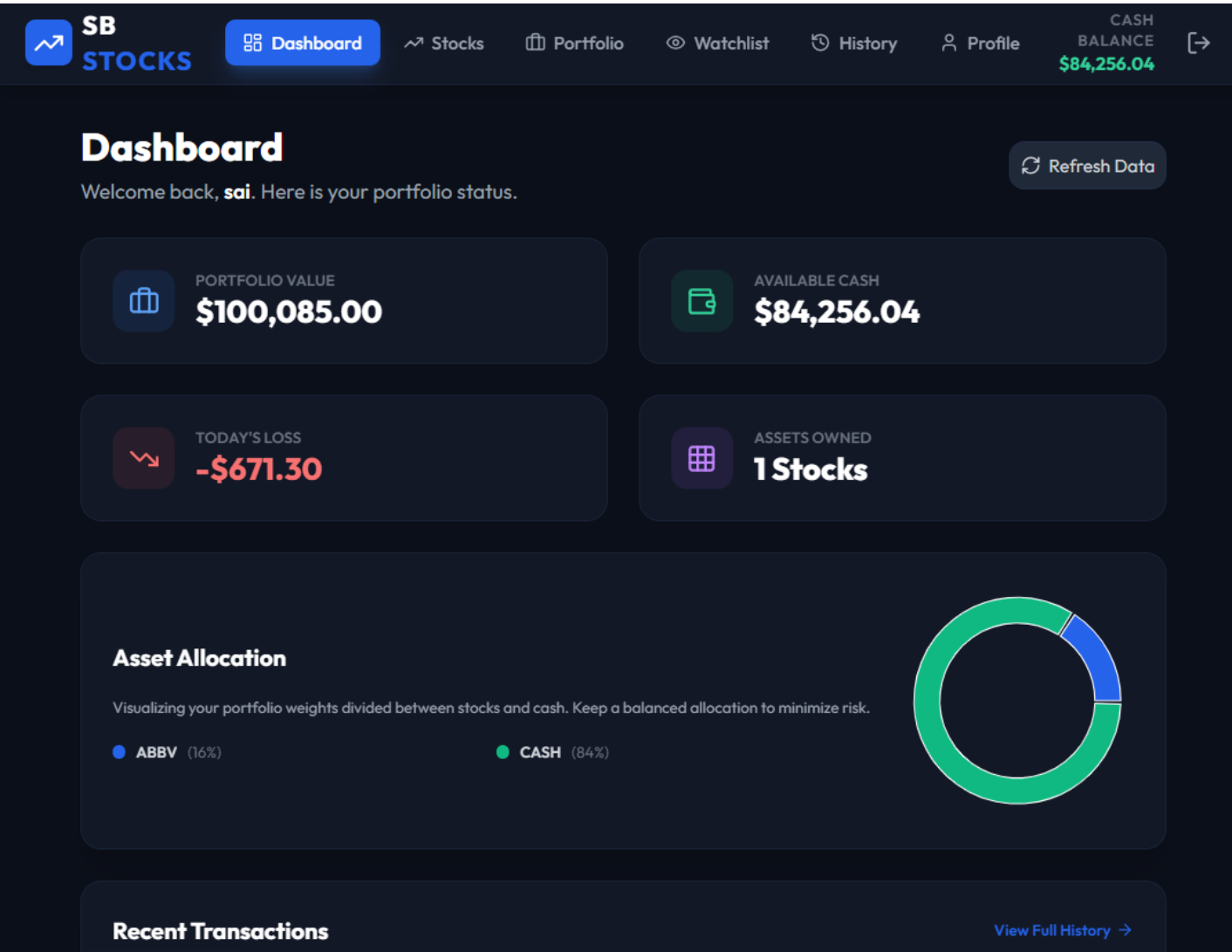
CONFIRM PASSWORD

 Re-enter password

Create Account →

Already have an account? [Sign In](#)

Dashboard:



STOCK MARKETS:



SB

STOCKS

Dashboard

Stocks

Portfolio

Watchlist

History

Profile

CASH BALANCE

\$84,256.04

→

Stock Markets

Search, filter, and monitor live assets for paper trading.

Q

Search ticker symbol or company name... (Press Enter)

SECTOR:

All

↓

↑↓ SORT BY:

Symbol (A-Z)

↓

AAPL

NASDAQ

✓

Apple Inc.

Technology

PRICE

\$166.40

+2.19%

View Details

ABBV

NASDAQ

👁

AbbVie Inc.

Healthcare

PRICE

\$162.26

-3.63%

View Details

ABT

NASDAQ

👁

Abbott Laboratories

Healthcare

PRICE

\$107.69

-3.01%

View Details

ADBE

NASDAQ

✓

Adobe Inc.

Technology

PRICE

\$433.44

+3.54%

View Details

ADM

NASDAQ

👁

Archer-Daniels-Midland Company

Consumer Staples

PRICE

\$57.11

+3.57%

View Details

ADP

NASDAQ

👁

Automatic Data Processing, Inc.

Industrials

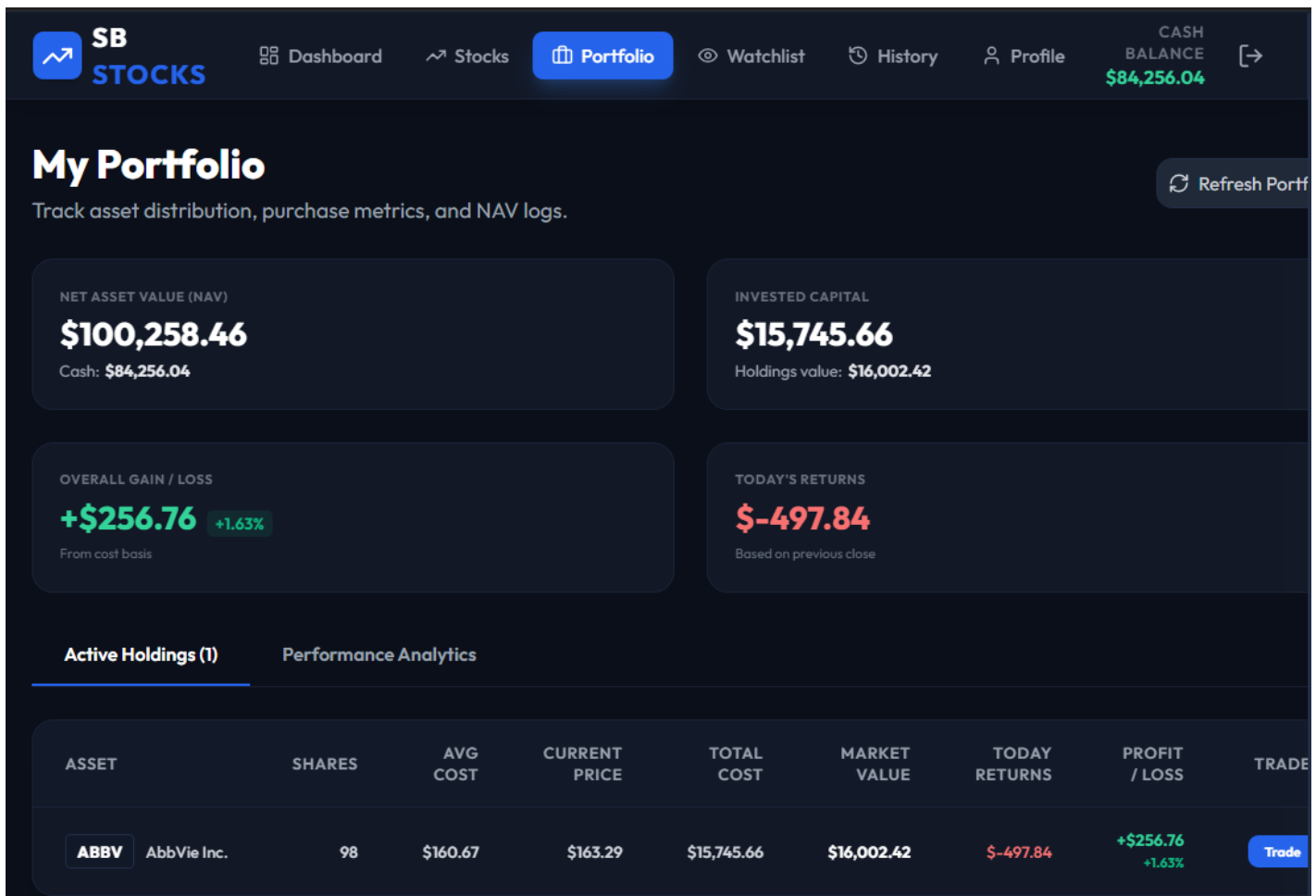
PRICE

\$247.85

-0.30%

View Details

PORTFOLIO:



## CONCLUSION

The SB Stocks project provides a complete stock trading simulation platform that allows users to learn and practice trading in a safe and risk-free environment. The application enables users to explore stock market data, perform virtual buying and selling operations, manage portfolios, track investments, and analyze transaction history effectively.

By integrating React.js for frontend development, Node.js and Express.js for backend services, and MongoDB for database management, the system delivers a responsive, scalable, and efficient trading experience. Features such as secure authentication, watchlist management, portfolio tracking, and transaction monitoring make the platform more interactive and useful for users.

Overall, SB Stocks helps beginners understand stock market concepts and allows traders to test investment strategies without financial risk. The project demonstrates the successful implementation of a full-stack MERN application with practical features that improve financial learning and decision-making skills.